

120+ Tricky Python Guess-the-Output Questions (with Explanations)

Curated from commonly asked Python tricky interview patterns (especially output-driven and behavior-driven questions).

Section 1: Identity, Mutability, and Defaults (Q1-15)

Q1

```
a = [1, 2]
b = [1, 2]
print(a == b, a is b)
```

Answer: True False **Explanation:** Same value, different object identity.

Q2

```
a = [1, 2]
b = a
print(a is b)
```

Answer: True

Q3

```
def f(x=[]):
    x.append(1)
    return x

print(f())
print(f())
```

Answer: [1] then [1, 1] **Explanation:** Mutable default created once.

Q4

```
def f(x=None):
    if x is None:
        x = []
    x.append(1)
    return x

print(f())
print(f())
```

Answer: [1] then [1]

Q5

```
x = [[0]] * 3
x[0][0] = 9
print(x)
```

Answer: [[9], [9], [9]]

Q6

```
a = [1, 2]
b = a
a = a + [3]
print(a, b)
```

Answer: [1, 2, 3] [1, 2]

Q7

```
a = [1, 2]
b = a
a += [3]
print(a, b)
```

Answer: [1, 2, 3] [1, 2, 3]

Q8

```
print(bool([]), bool([0]))
```

Answer: False True

Q9

```
print([] == False, [] is False)
```

Answer: False False

Q10

```
x = 256
y = 256
print(x is y)
```

Answer: Usually True (implementation detail).

Q11

```
x = 257
y = 257
print(x is y)
```

Answer: Implementation-dependent, do not rely on `is` for numeric equality.

Q12

```
print(None == None, None is None)
```

Answer: True True

Q13

```
s = "hello"
print(s[::-1])
```

Answer: olleh

Q14

```
print([1, 2, 3][-1])
```

Answer: 3

Q15

```
print("abdc1321".isalnum(), "xyz@123$".isalnum())
```

Answer: True False

Section 2: Scope, Closures, and Functions (Q16-30)

Q16

```
x = 10

def f():
    x = 20
    print(x)

f()
print(x)
```

Answer: 20 then 10

Q17

```
x = 10

def f():
    global x
    x = 20

f()
print(x)
```

Answer: 20

Q18

```
funcs = []
for i in range(3):
    funcs.append(lambda: i)

print([f() for f in funcs])
```

Answer: [2, 2, 2]

Q19

```
funcs = []
for i in range(3):
    funcs.append(lambda i=i: i)

print([f() for f in funcs])
```

Answer: [0, 1, 2]

Q20

```
def f(a, b=2, c=3):
    return a + b + c

print(f(1), f(1, 5), f(1, c=10))
```

Answer: 6 9 13

Q21

```
def f(*args):
    return sum(args)
```

```
print(f(1, 2, 3))
```

Answer: 6

Q22

```
def f(**kwargs):  
    print(sorted(kwargs.items()))  
  
f(b=2, a=1)
```

Answer: [('a', 1), ('b', 2)]

Q23

```
def outer(x):  
    def inner(y):  
        return x + y  
    return inner  
  
print(outer(10)(5))
```

Answer: 15

Q24

```
def f():  
    try:  
        return 1  
    finally:  
        return 2  
  
print(f())
```

Answer: 2

Q25

```
def f():  
    try:  
        print("A")  
    finally:  
        print("B")  
  
f()
```

Answer: A then B

Q26

```
print((lambda x: x * 2)(5))
```

Answer: 10

Q27

```
def f(a, b, /, c):  
    return a + b + c  
  
print(f(1, 2, c=3))
```

Answer: 6

Q28

```
def f(a, *, b):  
    return a + b  
  
print(f(2, b=3))
```

Answer: 5

Q29

```
def f(x):  
    x.append(4)  
  
arr = [1, 2, 3]  
f(arr)  
print(arr)
```

Answer: [1, 2, 3, 4]

Q30

```
def f(x):  
    x = x + [4]  
  
arr = [1, 2, 3]  
f(arr)  
print(arr)
```

Answer: [1, 2, 3]

Section 3: Comprehensions, Iterators, Generators (Q31-45)

Q31

```
print([x*x for x in range(4)])
```

Answer: [0, 1, 4, 9]

Q32

```
print({x: x*x for x in [1, 2, 3]})
```

Answer: {1: 1, 2: 4, 3: 9}

Q33

```
print([x for x in range(5) if x % 2])
```

Answer: [1, 3]

Q34

```
nums = [1, 2, 3]
it = iter(nums)
print(next(it), next(it))
```

Answer: 1 2

Q35

```
def gen():
    yield 1
    yield 2

g = gen()
print(next(g), next(g))
```

Answer: 1 2

Q36

```
def gen():
    for i in range(3):
        yield i

print(list(gen()))
```

Answer: [0, 1, 2]

Q37

```
print(sum(i for i in range(4)))
```

Answer: 6

Q38

```
print(any([0, "", None, 5]))
```

Answer: True

Q39

```
print(all([1, 2, 3]))
```

Answer: True

Q40

```
print(all([1, 0, 3]))
```

Answer: False

Q41

```
pairs = [(1, 2), (3, 4)]  
print([a+b for a, b in pairs])
```

Answer: [3, 7]

Q42

```
print([x for x in "abc"])
```

Answer: ['a', 'b', 'c']

Q43

```
arr = [1, 2, 3]  
print(arr[:2])
```

Answer: [1, 3]

Q44

```
print(list(zip([1,2], ['a','b','c'])))
```

Answer: [(1, 'a'), (2, 'b')]

Q45

```
print(dict([("a", 1), ("b", 2)]))
```

Answer: {'a': 1, 'b': 2}

Section 4: Classes, Dunder, and OOP Gotchas (Q46-60)

Q46

```
class A:
    x = 1

print(A.x)
```

Answer: 1

Q47

```
class A:
    x = []

a1 = A()
a2 = A()
a1.x.append(1)
print(a2.x)
```

Answer: [1] **Explanation:** Class variable shared by instances.

Q48

```
class A:
    def __init__(self):
        self.x = []

a1 = A()
a2 = A()
a1.x.append(1)
print(a2.x)
```

Answer: []

Q49

```
class A:
    def __str__(self):
        return "A"

print(A())
```

Answer: A

Q50

```
class A:
    def __repr__(self):
        return "R"

print(A())
```

Answer: R

Q51

```
class A:
    def __len__(self):
        return 0

print(bool(A()))
```

Answer: False

Q52

```
class A:
    pass

obj = A()
obj.x = 10
print(obj.x)
```

Answer: 10

Q53

```
class A:
    __x = 5

print(hasattr(A, '__x'), hasattr(A, '_A__x'))
```

Answer: False True

Q54

```
class A:
    def f(self):
        return "A"

class B(A):
    def f(self):
        return "B"

print(B().f())
```

Answer: B

Q55

```
class A:
    def f(self):
        return 1

class B(A):
    pass

print(B().f())
```

Answer: 1

Q56

```
class A:
    def __init__(self):
        print("A")

class B(A):
    def __init__(self):
        super().__init__()
        print("B")

B()
```

Answer: A then B

Q57

```
class A:
    @staticmethod
    def f():
```

```
        return 1

print(A.f())
```

Answer: 1

Q58

```
class A:
    @classmethod
    def f(cls):
        return cls.__name__

print(A.f())
```

Answer: A

Q59

```
class A:
    def __call__(self, x):
        return x + 1

print(A()(5))
```

Answer: 6

Q60

```
class A:
    pass

print(isinstance(A(), A))
```

Answer: True

Section 5: Exceptions, Context Managers, and Control Flow (Q61-75)

Q61

```
def f():
    try:
        return 1
    finally:
        print("F")

print(f())
```

Answer: F then 1

Q62

```
try:
    1 / 0
except ZeroDivisionError:
    print("Z")
```

Answer: Z

Q63

```
try:
    int("x")
except ValueError:
    print("V")
finally:
    print("F")
```

Answer: V then F

Q64

```
for i in range(3):
    if i == 1:
        continue
    print(i, end="")
```

Answer: 02

Q65

```
for i in range(3):
    if i == 1:
        break
    print(i, end="")
```

Answer: 0

Q66

```
for i in range(2):
    pass
else:
    print("done")
```

Answer: done

Q67

```
for i in range(2):  
    break  
else:  
    print("done")
```

Answer: No output.

Q68

```
class C:  
    def __enter__(self):  
        print("E")  
        return self  
    def __exit__(self, exc_type, exc, tb):  
        print("X")  
  
with C():  
    print("B")
```

Answer: E then B then X

Q69

```
class C:  
    def __enter__(self):  
        return self  
    def __exit__(self, exc_type, exc, tb):  
        print(exc_type.__name__)  
        return True  
  
with C():  
    1 / 0  
print("ok")
```

Answer: ZeroDivisionError then ok

Q70

```
try:  
    raise RuntimeError("x")  
except Exception as e:  
    print(type(e).__name__)
```

Answer: RuntimeError

Q71

```
def f():  
    try:  
        raise ValueError("a")  
    except ValueError:  
        return "handled"  
  
print(f())
```

Answer: handled

Q72

```
def f(x):  
    match x:  
        case 1:  
            return "one"  
        case _:  
            return "other"  
  
print(f(1), f(2))
```

Answer: one other

Q73

```
x = 1  
if x:  
    print("T")  
else:  
    print("F")
```

Answer: T

Q74

```
print("A" if 0 else "B")
```

Answer: B

Q75

```
print([i for i in range(5) if i % 2 == 0])
```

Answer: [0, 2, 4]

Section 6: Threads, Asyncio, GIL, and Parallelism (Q76-95)

Q76

```
import threading

def w():
    print("T", end="")

t = threading.Thread(target=w)
t.start()
t.join()
print("M", end="")
```

Answer: TM

Q77

```
import threading

x = 0
def inc():
    global x
    for _ in range(10000):
        x += 1

t1 = threading.Thread(target=inc)
t2 = threading.Thread(target=inc)
t1.start(); t2.start()
t1.join(); t2.join()
print(x)
```

Answer: Often 20000 , but race-prone pattern in principle.

Q78

```
import threading

lock = threading.Lock()
x = 0
def inc():
    global x
    for _ in range(1000):
        with lock:
            x += 1

t1 = threading.Thread(target=inc)
t2 = threading.Thread(target=inc)
t1.start(); t2.start()
```



```
t1.join(); t2.join()  
print(x)
```

Answer: 2000

Q79

```
import asyncio  
  
async def main():  
    print("A")  
  
asyncio.run(main())
```

Answer: A

Q80

```
import asyncio  
  
async def f():  
    await asyncio.sleep(0)  
    return 42  
  
print(asyncio.run(f()))
```

Answer: 42

Q81

```
import asyncio  
  
async def a():  
    await asyncio.sleep(0.01)  
    return "A"  
  
async def b():  
    await asyncio.sleep(0.01)  
    return "B"  
  
async def main():  
    r = await asyncio.gather(a(), b())  
    print(r)  
  
asyncio.run(main())
```

Answer: ['A', 'B']

Q82

```
import asyncio

async def main():
    t = asyncio.create_task(asyncio.sleep(0.01, result=7))
    print(await t)

asyncio.run(main())
```

Answer: 7

Q83

```
import asyncio

async def main():
    try:
        await asyncio.wait_for(asyncio.sleep(1), timeout=0.01)
    except asyncio.TimeoutError:
        print("timeout")

asyncio.run(main())
```

Answer: timeout

Q84

```
import concurrent.futures

with concurrent.futures.ThreadPoolExecutor(max_workers=2) as ex:
    fut = ex.submit(lambda: 40 + 2)
    print(fut.result())
```

Answer: 42

Q85

```
import concurrent.futures

with concurrent.futures.ProcessPoolExecutor(max_workers=1) as ex:
    fut = ex.submit(sum, [1, 2, 3])
    print(fut.result())
```

Answer: 6

Q86

```
import queue
```

```
q = queue.Queue()
q.put(1)
q.put(2)
print(q.get(), q.get())
```

Answer: 1 2

Q87

```
import threading

e = threading.Event()
print(e.is_set())
e.set()
print(e.is_set())
```

Answer: False then True

Q88

```
import threading

local = threading.local()
local.x = 1
print(local.x)
```

Answer: 1

Q89

```
import asyncio

async def main():
    async def w(i):
        await asyncio.sleep(0)
        return i
    print(await asyncio.gather(*(w(i) for i in range(3))))

asyncio.run(main())
```

Answer: [0, 1, 2]

Q90

```
import threading

print(threading.current_thread().name == "MainThread")
```

Answer: True

Q91

```
import asyncio

async def main():
    print(asyncio.get_running_loop() is not None)

asyncio.run(main())
```

Answer: True

Q92

```
import time
from functools import lru_cache

@lru_cache(maxsize=None)
def f(x):
    time.sleep(0.01)
    return x * x

print(f(3), f(3))
```

Answer: 9 9

Q93

```
from functools import lru_cache

@lru_cache(maxsize=2)
def f(x):
    return x

f(1); f(2); f(3)
print(f.cache_info().currsize)
```

Answer: 2

Q94

```
import asyncio

async def main():
    await asyncio.sleep(0)
    print("done")
```

```
asyncio.run(main())
```

Answer: done

Q95

```
import threading

def w():
    pass

t = threading.Thread(target=w)
print(t.is_alive())
t.start(); t.join()
print(t.is_alive())
```

Answer: False then False

Section 7: Collections, Itertools, and Functional Gotchas (Q96-110)

Q96

```
from collections import Counter

c = Counter("abca")
print(c["a"], c["z"])
```

Answer: 2 0

Q97

```
from collections import defaultdict

d = defaultdict(int)
d["x"] += 1
print(d["x"])
```

Answer: 1

Q98

```
from collections import deque

d = deque([1, 2, 3])
d.appendleft(0)
print(list(d))
```

Answer: [0, 1, 2, 3]

Q99

```
from collections import deque

d = deque([1, 2, 3], maxlen=3)
d.append(4)
print(list(d))
```

Answer: [2, 3, 4]

Q100

```
import heapq

h = [3, 1, 2]
heapq.heapify(h)
print(heapq.heappop(h))
```

Answer: 1

Q101

```
print(sorted(["aa", "b", "ccc"], key=len))
```

Answer: ['b', 'aa', 'ccc']

Q102

```
print(list(map(lambda x: x * 2, [1, 2, 3])))
```

Answer: [2, 4, 6]

Q103

```
print(list(filter(lambda x: x % 2, [1, 2, 3, 4])))
```

Answer: [1, 3]

Q104

```
from itertools import islice

print(list(islice(range(10), 2, 6)))
```

Answer: [2, 3, 4, 5]

Q105

```
from itertools import chain

print(list(chain([1, 2], [3], [])))
```

Answer: [1, 2, 3]

Q106

```
print(dict.fromkeys(["a", "b"], 0))
```

Answer: {'a': 0, 'b': 0}

Q107

```
print(list({3, 1, 2}))
```

Answer: Set iteration order is arbitrary.

Q108

```
d = {"a": 1, "b": 2}
print(list(d.keys()))
```

Answer: ['a', 'b'] **Explanation:** Dict preserves insertion order in modern Python.

Q109

```
d = {"x": 1}
print(d.get("y", -1))
```

Answer: -1

Q110

```
a = [1, 2, 3]
b = a.copy()
b[0] = 9
print(a, b)
```

Answer: [1, 2, 3] [9, 2, 3]

Section 8: Caches, Descriptors, and Production Patterns (Q111-120)

Q111

```

from functools import lru_cache

@lru_cache(maxsize=None)
def fib(n):
    if n < 2:
        return n
    return fib(n - 1) + fib(n - 2)

print(fib(10))

```

Answer: 55

Q112

```

cache = {}
cache.setdefault("x", []).append(1)
cache.setdefault("x", []).append(2)
print(cache)

```

Answer: {'x': [1, 2]}

Q113

```

cache = {}
cache["x"] = cache.get("x", 0) + 1
cache["x"] = cache.get("x", 0) + 1
print(cache["x"])

```

Answer: 2

Q114

```

from collections import OrderedDict

od = OrderedDict()
od["a"] = 1
od["b"] = 2
od.move_to_end("a")
print(list(od.keys()))

```

Answer: ['b', 'a']

Q115

```

from dataclasses import dataclass

@dataclass(frozen=True)
class Key:

```



```
x: int

d = {Key(1): "ok"}
print(d[Key(1)])
```

Answer: ok

Q116

```
class A:
    def __init__(self):
        self._x = 10
    @property
    def x(self):
        return self._x

print(A().x)
```

Answer: 10

Q117

```
class D:
    def __get__(self, obj, owner):
        return 42

class A:
    x = D()

print(A().x)
```

Answer: 42

Q118

```
import weakref

class A:
    pass

a = A()
r = weakref.ref(a)
print(r() is a)
```

Answer: True

Q119

```
import json

d = {"a": 1, "b": [2, 3]}
s = json.dumps(d, sort_keys=True)
print(isinstance(s, str))
```

Answer: True

Q120

```
print("Production note: use locks for shared mutable cache state and benchmark before over-optimizing.")
```

Answer: Prints the exact line.

Interview Tips

- Use `==` for value comparison and `is` only for identity/singletons like `None`.
 - Mention mutable default argument trap quickly and provide `None` fix.
 - For closure questions, call out late binding and default argument workaround.
 - For class variable questions, distinguish class attribute vs instance attribute.
 - In threading/asyncio questions, always call out ordering nondeterminism unless synchronized.
 - For cache questions, mention `lru_cache`, invalidation strategy, and thread-safety expectations.
 - In dict/set output questions, distinguish insertion order (dict) from arbitrary order (set).
 - For production interviews, mention when multiprocessing can bypass GIL for CPU-heavy tasks.
-

Companion: Regular Problems + Solutions (One-Stop)

For non-tricky, implementation-focused interview prep (collections, asyncio/threading, cache patterns, and DSA/Algo), use:

- `41_PYTHON_ONE_STOP_DSA_ALGO_REGULAR_PROBLEMS.md`